

Learning Sensor Sample-Reweighting for Dynamic Early-Exit Activity Recognition Via Meta Learning

Zenan Fu¹, Lei Zhang¹, Member, IEEE, Wenbo Huang², Dongzhou Cheng², Hao Wu²,
and Aiguo Song², Senior Member, IEEE

Abstract—During recent years, dynamic early-exit has provided a promising paradigm to improve the computational efficiency of deep neural networks by constructing multiple classifiers to let easy samples exit at shallow layers while avoiding redundant computations at deep exits, which has been seldom explored in the context of latency-aware human activity recognition (HAR) deployed on wearable devices. Particularly, most existing early-exit strategies have always treated all activity samples equally at each exit during training, which ignore such dynamic early-exit behavior at test-time, causing a potential mismatch between training and test. Intuitively, easy activity samples that often exit earlier at test-time should place more emphasis on the training loss of shallow classifiers, while hard activity samples should contribute more to the training loss of deep classifiers. To bridge this gap, this paper introduces a sample-reweighting approach for efficient activity inference, which employs a weight-predicting network to reweight the training loss of different activity samples at every exit. From a perspective of meta learning, a new optimization objective function is designed to jointly optimize both weight-predicting network and backbone network. We perform extensive experiments on three popular HAR benchmarks including UCI-HAR, WISDM, and UniMiB-SHAR, which demonstrate that while incorporating such test-time early-exit behavior into conventional training pipeline, it can consistently improve the accuracy-efficiency trade-offs under budgeted batch classification and anytime prediction patterns. Moreover, our approach has a natural advantage in handling class-imbalance HAR

problem. Detailed ablation studies, visualized illustrations, and real hardware deployment are provided to support our statement.

Index Terms—Activity recognition, wearable devices, meta learning, sensor data, dynamic early-exit.

I. INTRODUCTION

A. Background

THE last decade has seen major technological advancements in wearable sensors. Benefiting from its prominent advantages including smaller size, higher accuracy, and lower manufacturing prices, a broad range of miniaturized sensors has been embedded into portable devices such as phones and wristbands, which could provide smarter services [1]. Activity recognition (HAR) using on-body sensors has recently become a new research hotspot in ubiquitous computing scenario, which typically involves direct or indirect interaction between wearable devices and human actions. It could utilize popular machine learning algorithms to analyze or infer human activities through collected raw sensor data, leading to an increasing number of application situations like activity tracking, elderly-care, gesture analysis, health management, patient surveillance system, ambient assisted living (AAL), and human-computer interaction [2]. In particular, deep neural networks have garnered great attention, which could automatically extract the hierarchically abstracted features rather than exploring laborious handcrafted features from multivariate time series sensor data, hence achieving outstanding success in the field of human activity recognition (HAR). Despite strong representation power, deep neural networks always suffer from high computational cost, which hinders practical HAR deployment on budget-constrained wearable devices. How to accelerate activity inference using deep neural networks has become a challenging problem in the HAR area.

B. Limits of Prior ARTs

At present, there have already existed numerous popular solutions including lightweight network design, channel pruning, knowledge-distillation, weight quantization, and dynamic neural networks [3]. Among them, dynamic early-exit has gained considerable attention because of their favorable efficiency and flexibility [4], which can execute adaptive activity inference under varying computational budgets. Through embedding multiple intermediate classifiers into a given deep network, it enables different model components (e.g., convolutional layers [5]) to

Received 24 June 2024; revised 16 December 2024; accepted 27 December 2024. Date of publication 31 December 2024; date of current version 6 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62373194, in part by the Cultivating Plan Program for the Leader in Science and Technology of Yunnan Province, China under Grant 202005AC160005, and in part by the Ten Thousand Talent Plans for Young of Yunnan Province, China under Grant YNWR-QNBJ-2019-188. (Corresponding author: Lei Zhang.)

Zenan Fu, Lei Zhang, and Dongzhou Cheng are with the School of Electrical and Automation Engineering, Nanjing Normal University, Nanjing 210023, China (e-mail: leizhang@njnu.edu.cn).

Wenbo Huang is with the School of Computer Science and Engineering, Southeast University, Nanjing 210096, China.

Hao Wu is with the School of Information Science and Engineering, Yunnan University, Kunming 650500, China.

Aiguo Song is with the School of Instrument Science and Engineering, Southeast University, Nanjing 210096, China.

Code will be available at <https://github.com/FuZenan/LtSRW-for-EEAR>.

Digital Object Identifier 10.1109/JBHI.2024.3524252

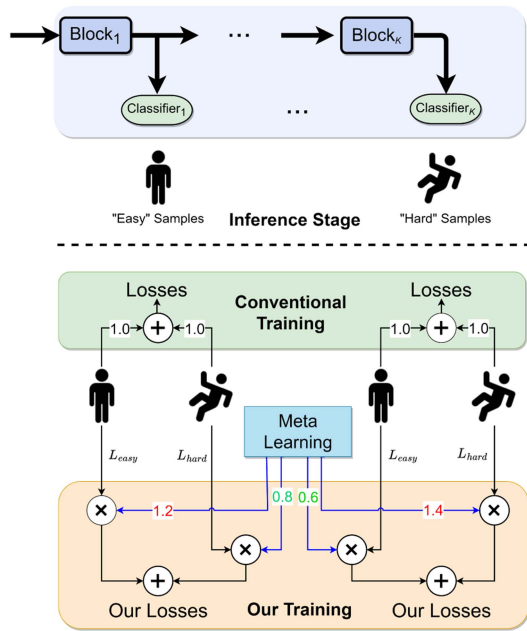


Fig. 1. An exemplar illustration of the proposed sample-reweighting strategy. During inference phase, both ‘Fall’ and ‘Standing’ having different difficulty levels are handled at different exits (top). However, the conventional training scheme (middle) equally treats all activity samples across all exits, causing a potential training-test data distribution mismatch. To mitigate this, our sample-reweighting scheme (bottom) encourages each exit to emphasize different activity samples by reweighting their training loss.

be conditionally skipped in a data-dependent manner, based on the difficulty level of input activity sample. If the predictive confidence at an early-exit point meets a predefined condition (i.e., exceeding certain confidence threshold), the forward-pass inference will be stopped in advance, and the redundant computation of later exits can be saved. Such an adaptive scheme can considerably accelerate activity inference without sacrificing accuracy, which can make predictions at earlier exits to save a substantial amount of computation for ‘easy’ activity samples, while handling the ‘hard’ activity samples at later exits, as illustrated in Fig. 1. Most existing early-exit works have typically focused on designing more sophisticated multi-exit structures, which generally adopts a naive training scheme to sum up the cross-entropy loss aggregated from all the classifiers. In such a scenario, both ‘hard’ and ‘easy’ activity samples would always contribute equally to the final loss of optimized objective, no matter where an activity sample would really exit (also see conventional training in Fig. 1). However, during test-time, it should be highlighted that multiple classifiers at varying exits should have essentially unequal capabilities, which are responsible for recognizing activity samples at diverse difficulty levels, hence leading to a potential discrepancy between training and test phases. Such training-test mismatch phenomenon has been long ignored in prior most works that apply the conventional training paradigm.

C. Research Motivation

In this work, we are mainly concerned with understanding about how deep models perform adaptive activity inference while the data distribution differs between training and deployment, which is a natural situation arising in many early-exit HAR

applications. At training time, all the classifiers require being optimized equally over all training samples; In the opposite, not all the classifiers can equally handle all types of activity data at test time. For example, given that the computational budget is very tight, earlier classifiers will only see ‘standing’ activity that is easy to process, while deeper classifiers will fail to see it (also see the top panel in Fig. 1) and instead place more emphasis on ‘fall’ that is relatively hard to recognize. To mitigate this discrepancy between training and test phases, this paper mainly aims to incorporate test-time dynamic behavior into training phase via adapting sample-aware weights over the total loss of multi-exits for efficient activity inference. This work is motivated by the fact that each classifier only needs to process a subset of its most confident activity samples, where the earlier shallow classifiers are often responsible for those predominant ‘easy’ activity samples, while the deeper classifiers at later exits are capable of handling less ‘hard’ samples. Intuitively, an ideal optimization objective is to encourage every classifier to highlight different activity samples with varying difficulty levels through reweighting their training loss.

D. Main Contribution

In practice, it still remains two main challenges when reweighting training loss of different activity samples at multiple exits: 1) Since there is no prior knowledge about where an activity sample will exit, it is difficult to make an early-exit decision at training time; 2) How to automatically set appropriate weights causes a non-trivial problem. To tackle the aforementioned challenges, from a viewpoint of meta learning, this paper introduces a weight-predicting network (abbreviated as WPN), we leverage the total loss obtained from multiple exits as an input to dynamically generate the appropriate weights imposed over various training samples at each exit for efficient activity inference. To achieve this goal, we define a new optimization objective function to guide such meta-learning procedure through jointly optimizing both the backbone network and WPN together, which is able to precisely imitate dynamic early-exit behavior to determine where different activity samples would exit at training-time. The meta objective of every classifier could be then utilized as the related loss only over these samples that will really exit from a given classifier. In summary, the main contributions of our work are three-fold:

First, to the best of our knowledge, this paper is the first attempt to address the train-test mismatch problem in dynamic early-exit networks, which has been seldom explored in the context of resource-efficient HAR deployed on low-cost wearable devices. An actual evaluation is executed on a real hardware device.

Second, in contrast to the naive training scheme, our especially designed meta objective can encourage each exit to have its own appropriate weights imposed over different activity samples for better accuracy-cost trade-offs in early-exit HAR scenario. The class imbalance scenario is also considered.

Finally, we evaluate the proposed approach on three HAR benchmarks including UCI-HAR [6], WISDM [7], and UniMiB-SHAR [8], which demonstrate that it can consistently improve accuracy-cost trade-off for recent state-of-the-art backbone networks. Detailed ablation studies and visual analyses are provided.

The rest of this paper is organized as follows. Section II reviews related HAR literatures. Section III presents our meta-learning based activity recognition algorithm. Section IV

reports our main results, ablation studies, visualizing illustrations, realistic hardware deployment, and cross-domain evaluations. Finally, we conclude this paper in Section V.

II. RELATED WORK

Over the past ten years, extensive research efforts have been devoted to deep learning models, which achieves outstanding success in wearable-based human activity recognition situations. For instance, Yang et al. [9], Zeng et al. [10] and Ronao et al. [11] have at an earliest time exploited convolutional neural networks (CNNs) as backbone architecture to tackle HAR tasks. Hammerla et al. [12] have conducted a systematic exploration in applying convolutional and recurrent neural networks (RNNs) in HAR area. Ma et al. [6] have presented a powerful network named AttnSense by embedding an attention mechanism into a hybrid model built on CNN with Gated Recurrent Units (GRUs), which could capture the spatial-temporal dependencies from multivariate time series sensor signals for HAR. Jiang et al. [13] have proposed to reorganize multivariate accelerometers and gyroscopes signal sequences into two-dimensional input activity images, from which deep CNNs can automatically learn optimal features for activity recognition. Münzner et al. [14] have explored how to optimally fuse multimodal sensor signals while applying CNNs for activity recognition, which shows that both late and hybrid fusion strategies are superior to early fusion. Al-qaness et al. [15] have designed a novel deep learning model named Multi-ResAtt, which embeds a RNN with attention into multi-level residual networks to extract hierarchical features for activity recognition. Chen et al. [2] have recently provided a detail literature survey about deep learning for sensor-based human activity recognition. Despite remarkable success, these high-performance deep models have been typically accompanied by high computational cost. Since low latency always influence optimal use experience in real-world HAR situation, it still remains an essential challenge how to accelerate activity inference on resource-constrained wearable devices, especially under varying computational budgets, which has not yet gotten enough attention in ubiquitous HAR computing scenario.

To address above challenge, now there have already existed numerous popular lightweight designs employed to reduce the computational complexity of deep learning models in latency-aware HAR situations. For example, Luo et al. [16] have introduced a binarized deep neural network named Binary Dilated DenseNet, which is able to preserve ideal accuracy while supporting low-latency activity inference well at mobile devices. Liang et al. [17] have proposed to place channel pruning and tensor decomposition together for accelerating activity inference, which can exploit a collaborative compression strategy to reduce the on-device inference latency. Chen et al. [18] have introduced a novel knowledge-distilling strategy through applying a small shallow network (i.e., student) to mimic a large neural network (i.e., teach), where the small student network can be then deployed to mobile platforms, yielding faster activity inference. However, the aforementioned works are essentially static, which could not well adapt to a large variety of dynamic computing environments (e.g., a mobile phone works at low-power mode) or varying hardware platforms (e.g., at present, there have been at least 25000 types of mobile device belonging to difference brands), thereby leading to recent emergence of early-exit paradigm [19]. Lattanzi et al. [19] have at the earliest time explored the usage of early exit in their work ‘Do we need

early exit networks in human activity recognition?’, which provides an empirical analysis of accuracy–latency trade-offs across a wide range of HAR applications. In addition, they also analyze the energy consumption or sustainability while such early-exit networks are deployed to real-world HAR applications [20]. Seon et al. [21] have employs a LSTM-based policy module to control ‘stop or forward’, achieving better trade-off between accuracy and cost in activity recognition. Yang et al. [22] have proposed a dynamic sampling-rate network, which combines with a novel subsampling technique with conditional early-exit strategy to perform adaptive activity inference under varying computational budgets. Daghero et al. [23] have combined such dynamic networks with sub-byte or mixed-precision quantization, which can strike remarkable trade-offs between accuracy and memory footprint. In summary, most existing early-exit HAR literatures have mainly focus on the naïve training strategy, which always treats equally all activity samples at training-time. They potentially ignore an important case: different activity samples should have unequal importance, which make the training-test data distribution be highly divergent at different exits. At training-time, one usually could not precisely estimate the sample importance in advance due to lacking prior knowledge. Different from previous most works, the main goal of this paper is to establish such loss-weight mapping relation by applying an idea of meta learning to predict proper sample weights for different activity samples at multiple exits, which can find better accuracy-latency trade-offs in a general early-exiting HAR scenario, with no need of any assumption on the dataset.

III. METHOD

A. Preliminaries

Multi-exit networks: Multi-exit networks are typically comprised of multiple intermediate classifiers attached to a backbone network, offering the early predicted outputs during inference stage based on predefined exit policies. Without loss of generality, we typically add $K-1$ early-exit points into a deep neural network, so that along with the forward-pass pipeline, they can collectively form a multi-exit network with total K exits. Given that an activity sample $\mathbf{x}$ is fed into the aforementioned k -exit network, its output can be written as:

$$y^{(k)} = \arg \max_c p_c^{(k)} = \arg \max_c f_c^{(k)}(\mathbf{x}; \Theta_f^{(k)}),$$

$$k = 1, 2, \dots, K,$$
(1)

where $p_c^{(k)}$ is the probability of a specific activity class c predicted by the model while exiting at the k th exit, which can be simply calculated by using the softmax function. $\arg \max_c p_c^{(k)}$ indicates the most likely predicted value for the given activity sample at every exit. $f^{(k)}(\cdot; \Theta_f^{(k)})$ denotes the k th subnetwork f with parameter $\Theta_f^{(k)}$. As the deeper subnetworks $\Theta_f^{(k_m)}$, ($m > 1$) may include all of the prior networks $\Theta_f^{(k_n)}$, ($m > n > 1$) except for these attached classifiers, different depth subnetworks would naturally share their weights in a nested way.

Dynamic early exiting: There have already existed numerous works focusing on dynamic early-exit models for image classification tasks, among which the most classic and popular solution is to execute early-exit based on confidence threshold [20], [24]. For an activity sample, by comparing the predicted confidence

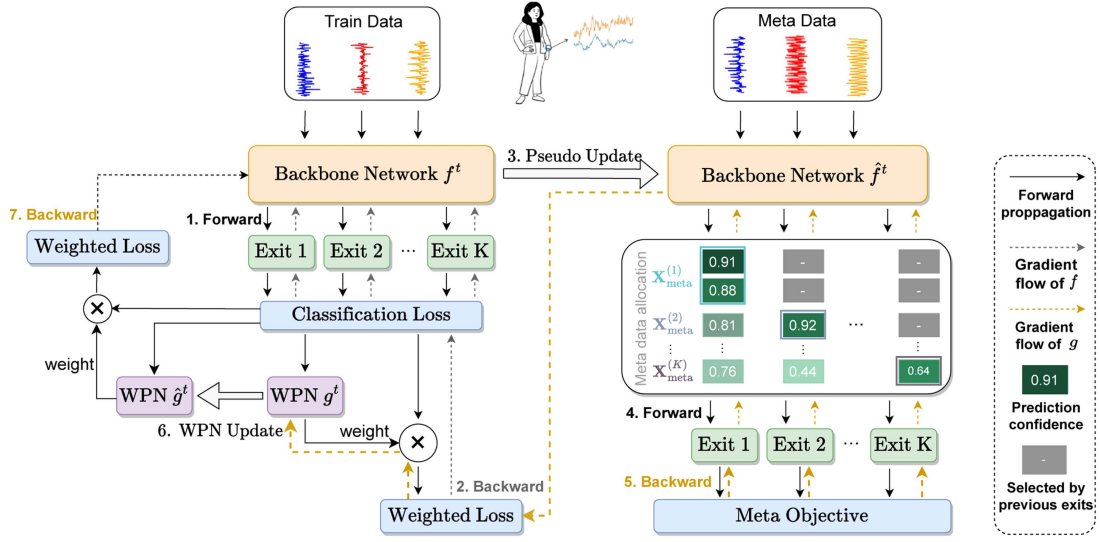


Fig. 2. Updates of the backbone model f and the WPN g in detail. It consists of 3 steps: 1) First, we calculate the weighted classification loss (5) using the training samples. This loss computation directs the pseudo-update process for the backbone network f^t , following Steps 1, 2, and 3. 2) A meta objective is computed using $\hat{f}^t$ on meta samples (6), the meta objective is calculated to update the WPN g^t (Steps 4, 5 and 6). 3) f^t is updated using the new weights predicted by g^{t+1} (Step 7).

$\max_c p_c^{(k)}$ at the k th exit to a predefined threshold (ϵ_k), deep models are able to from shallow to deep sequentially decide whether to exit current forward-pass behavior based on its difficulty level. The final output prediction may be formulated as:

$$y = y^{(k)}, \text{ if } \max_c p_c^{(k)} \geq \epsilon_k, \text{ and } \max_c p_c^{(j)} < \epsilon_j, \quad \forall j < k, k < K, \quad (2)$$

in which the predictions $y^{(k)}$ could be sequentially produced until the criterion in current equation is satisfied (i.e., surpassing the predefined confidence) or the last exit is reached. One can adaptively adjust the threshold $k(\epsilon_k)$ over each exit based on the practically available computational budget.

Conventional training scheme: Most existing works [24] generally apply a naïve training strategy to directly minimize a cumulative loss function, which aggregate the loss of all the samples equally across all exits:

$$\begin{aligned} \mathcal{L} &= \sum_{k=1}^K \frac{1}{N} \sum_{i=1}^N l_i^{(k)} \\ &\triangleq \sum_{k=1}^K \frac{1}{N} \sum_{i=1}^N \text{CE}(f^{(k)}(\mathbf{x}_i; \Theta_f^{(k)}), y_i), \end{aligned} \quad (3)$$

where CE represents standard cross-entropy loss, while N denotes the number of total activity samples.

B. Dynamic Early-Exit Based on Sample-Reweighting

In this part, we first describe the sample-reweighting mechanism, and then formulate the designed meta-learning objective. The detailed optimization algorithm procedure is illustrated in Algorithm 1. Fig. 2 provides an overall HAR framework of the training pipeline based on meta learning.

Sample-reweighting by WPN: From (2), we can see that during inference, different activity samples are dynamically allocated to multiple exits based on their predicted confidence. However, during training, the same optimization objective is equally employed over all activity samples for all these exits, as shown in (3), which would lead to contradiction between training and test phases. To bridge this gap, we propose to modify the training objective through reweighting the losses:

$$\mathcal{L} = \sum_{k=1}^K \frac{1}{N} \sum_{i=1}^N w_i^{(k)} l_i^{(k)}. \quad (4)$$

Weight-predicting network: Since it is difficult to establish the function mapping from input activity samples to weight due to lacking prior knowledge about where a given sample will exit, we propose a weight-predicting network (WPN) built as an MLP with one hidden layer to automatically learn the weight $w_i^{(k)}$ from an input $\mathbf{x}_i$: $\mathbf{w}_i = [w_i^{(1)}, w_i^{(2)}, \dots, w_i^{(K)}] = g(\mathbf{l}_i; \Theta_g)$, where $\mathbf{l}_i = [l_i^{(1)}, l_i^{(2)}, \dots, l_i^{(K)}]$ represents the training loss for sample $\mathbf{x}_i$ at K exits, and Θ_g denotes the parameters of the WPN. We apply a meta-learning principle to jointly update the backbone parameters Θ_f and the WPN parameters Θ_g . It is worth noting that the WPN only involves training phase, which would not incur extra computational burden at inference stage.

The meta-learning objective. Weighted classification loss: To mitigate the training-test mismatch, we bring the test-time dynamic behavior into training phase through constructing a new optimization objective function. Based on our weighting scheme denoted as Θ_g , we can formulate the optimization objective of the backbone model parameter Θ_f as:

$$\begin{aligned} \Theta_{\hat{f}} &= \arg \min_{\Theta_f} \mathcal{L}_{\text{train}}(\Theta_f, \Theta_g) \\ &\triangleq \arg \min_{\Theta_f} \sum_{k=1}^K \frac{1}{N} \sum_{i: \mathbf{x}_i \in \mathbf{X}_{\text{train}}} g^{(k)}(\mathbf{l}_i; \Theta_g) \cdot l_i^{(k)}(\Theta_f), \end{aligned} \quad (5)$$

Algorithm 1: The Meta-Learning Algorithm.

Require: Training data $\mathbf{D}(\mathbf{X}, \mathbf{y})$, batch size B , interval I , budget controller q , iteration T .

Optimization Objective: Backbone model f with parameters Θ_f .

for $t = 0$ **to** $T - 1$ **do**

 Split $\mathbf{D}\{\mathbf{X}, \mathbf{y}\}$ into $\text{MiniBatch}(D, B)$.

 Split $\mathbf{B}\{\mathbf{X}, \mathbf{y}\}$ into $\{\mathbf{X}_{\text{train}}, \mathbf{X}_{\text{meta}}, \mathbf{y}_{\text{train}}, \mathbf{y}_{\text{meta}}\}$.

if $t \% I = 0$ **then**

 Update f to a pseudo network with parameters Θ_f^t by (7).

 Split $\mathbf{X}_{\text{meta}}, \mathbf{y}_{\text{meta}}$ to $\{\mathbf{X}_{\text{meta}}^{(1)}, \dots, \mathbf{X}_{\text{meta}}^{(K)}\}$ and $\{\mathbf{y}_{\text{meta}}^{(1)}, \dots, \mathbf{y}_{\text{meta}}^{(K)}\}$ based on q .

 Update WPN g with parameters Θ_g^t to Θ_g^{t+1} by (8).

end if

 Update Θ_f^{t+1} by (9).

end for

where $l_i^{(k)}(\Theta_f)$ is the cross-entropy loss generated by the original network on the samples, and $g^{(k)}(l_i; \Theta_g)$ represents the weights predicted by our WPN, while $\mathbf{X}_{\text{train}}$ denotes the training set. Referring to previous literature [25], we further adjust the output of our WPN by applying a scaling factor denoted as δ where $0 < \delta < 1$, which is responsible for creating a perturbation $\tilde{\mathbf{w}} \in [-\delta, \delta]^{B \times K}$, with B denoting the batch size. This perturbation $\tilde{\mathbf{w}}$ is then normalized, which ensures that the sum of its elements is equal to 0. The resulting weight applied to the loss is finally calculated as $\mathbf{w} = 1 + \tilde{\mathbf{w}}$.

Meta data allocation: Our sample-reweighting mechanism aims to enhance model performance for dynamic activity inference. To this end, we define the optimization objective for the WPN g , and simulate the early-exit process on a meta set $\mathbf{X}_{\text{meta}}$ to obtain meta samples exiting at various exits by introducing a budget tuning variable q , which is in charge of controlling the number of activity samples being output at each exit. To be specific, the number of samples that exit at the k th exit can be written as: $N_k = \frac{q^k}{\sum_{k=1}^K q^k} \times N_{\text{samples}}$, $q > 0$, where $q = 1$ means that the output quantities at different exits are equivalent. On the other hand, while $q > 1$, it indicates that more activity samples would be output from deeper exits, accompanied by increased computation. During training stage, we usually adjust the variable q on the hold-out validation set, and instead keep it fixed during the meta-optimization process. During test stage, by varying q from 0 to 2, we may execute dynamic activity inference within a broad range of computational budgets, as indicated by accuracy-cost curves plotted in later Fig. 4 (Section IV). For a given q , Fig. 2 (right) presents a detailed procedure of obtaining $\mathbf{X}_{\text{meta}}^{(k)}$: every exit selects a handful of its most confident N_k activity samples that have not been handled by previous classifiers. It should be emphasized that at the last exit, $\mathbf{X}_{\text{meta}}^{(K)}$ must contain all the previously unchosen samples.

Meta objective for dynamic early-exit: For a specific exit, the meta objective is to only impose the classification loss on these activity samples that really exit at current exit, rather than handling all meta samples having diverse complexity. In this manner, through aggregating the meta objectives from total K exits, we can formulate the overall meta objective as

follows:

$$\begin{aligned} \Theta_g &= \arg \min_{\Theta_g} \mathcal{L}_{\text{meta}}(\Theta_f(\Theta_g)) \\ &\triangleq \arg \min_{\Theta_g} \sum_{k=1}^K \frac{1}{N_k} \sum_{j: \mathbf{x}_j \in \mathbf{X}_{\text{meta}}^{(k)}} l^{(k)}(\mathbf{x}_j, \mathbf{y}_j; \Theta_f(\Theta_g)). \end{aligned} \quad (6)$$

The optimization method: Generally speaking, meta learning seeks to train models on diverse tasks or datasets based on data assumptions, where the model will learn from a meta-dataset and generalizes its knowledge to new functions with only a few examples. In our case, we jointly optimize the backbone model and the WPN in a meta-learning way. Fig. 2 (also see the right panel) presents a detailed optimization pipeline based on our meta learning, where both the backbone weights Θ_f denoted in (5) and the WPN weights Θ_g denoted in (6) can be jointly optimized via an online scheme. Previous most meta-learning works generally require a standalone meta-set for loss-reweighting, that is comprised of clean labels and class-balanced data [26]. Different from them, we simply reuse the training data as our meta dataset. To be more specific, for a given iteration t , we divide the input mini-batch into two equal parts: a training batch $\mathbf{X}_{\text{train}}$ and a meta batch $\mathbf{X}_{\text{meta}}$, where the former is in charge of forming the classification loss in (5) while training the model weights Θ_f , and the latter is then in charge of calculating the meta objective in (6) while training the WPN weights Θ_g . The overall optimized procedure includes three main stages.

Pseudo update of the backbone network may be performed via $\mathbf{X}_{\text{train}}$:

$$\Theta_f^{(t+1)}(\Theta_g) = \Theta_f^t - \alpha \frac{\partial \mathcal{L}_{\text{train}}(\Theta_f, \Theta_g)}{\partial \Theta_f} \bigg|_{\Theta_f^t}, \quad (7)$$

in which α denotes the learning rate. Since $\Theta_f^{(t+1)}$ can be seen as a function of Θ_g , the main goal of this pseudo update is to construct the computational graph for optimizing Θ_g later. Fig. 2 illustrates the gradient flow during this pseudo update, as indicated by gray dashed lines. **Update of the WPN** is then realized through optimizing meta objective to mimic the early-exit procedure on $\mathbf{X}_{\text{meta}}$, which is divided into total K subsets without overlapping, i.e., $\mathbf{X}_{\text{meta}} = \{\mathbf{X}_{\text{meta}}^{(1)} \cup \mathbf{X}_{\text{meta}}^{(2)} \cup \dots \cup \mathbf{X}_{\text{meta}}^{(K)}\}$, where $\mathbf{X}_{\text{meta}}^{(k)}$ covers the meta samples that should be output at the k th exit based on the early-exit condition in (2). Fig. 2 illustrates this procedure. After receiving the pseudo updated backbone network based on the partition of the meta-data, we can update WPN weights as follows:

$$\Theta_g^{t+1} = \Theta_g^t - \beta \frac{\partial \mathcal{L}_{\text{meta}}(\Theta_f^{(t+1)}(\Theta_g))}{\partial \Theta_g} \bigg|_{\Theta_g^t}, \quad (8)$$

where $\mathcal{L}_{\text{meta}}(\Theta_f^{(t+1)}(\Theta_g))$ is equivalent to the summation of the classification loss aggregated from every exit based on its allocated meta data (also see (6)), where β denotes the learning rate. Fig. 2 illustrates the gradient flow used to update Θ_g , as indicated by the golden dashed lines.

Update of the backbone network is finally performed on basis of the updated Θ_g^{t+1} :

$$\Theta_f^{t+1} = \Theta_f^t - \alpha \frac{\partial \mathcal{L}_{\text{train}}(\Theta_f, \Theta_g^{t+1})}{\partial \Theta_f} \bigg|_{\Theta_f^t}. \quad (9)$$

The whole learning procedure is summarized in Algorithm 1. As highlighted by red font in (6), the meta objective can encourage every exit to only attend to a subset of activity samples that are most likely associated with it, where earlier exits would emphasize more ‘easy’ samples, while later ones would focus on less challenging cases. Section IV will further detail how it promotes such dynamically-early-exit behavior for efficient activity inference.

IV. EXPERIMENTS

A. Experimental Setup

Datasets: To comprehensively evaluate the proposed approach, we perform extensive experiments on the three publicly available human activity recognition benchmark datasets, including UCI-HAR [6], WISDM [7], and UniMiB-SHAR [8], which contain a wide range of human activities recorded by various sensor modalities at different sampling rates. To be specific, the UCI-HAR dataset covers human activities including Standing, Walking, Walking Downstairs, Walking Upstairs, Sitting, and Laying, recorded at a constant frequency of 50 Hz by triaxial accelerometer and triaxial gyroscope sensors embedded into a Samsung Galaxy S II phone. The WISDM dataset captures daily activities of living (ADLs) at a fixed frequency of 20 Hz by applying a triaxial accelerometer embedded in Android phones such as Nexus One and HTC Hero. The UniMiB-SHAR dataset includes 9 types of ADLs (e.g. Going downstairs, Jumping, Standing up from laying, Sitting down) and 8 types of falls (e.g., Falling with protection strategies, Generic falling backward, Syncope), recorded at a constant frequency of 50 Hz by a Bosh BMA220 accelerometer integrated into a Samsung Galaxy Nexus I9250. The raw sensor signals need to first go through a standard preprocessing pipeline such as data normalization, noise filtering, handling missing values, which are then divided into a series of fixed-length sliding windows with an overlap rate as model input. For fair comparisons, we still adopt the same data preprocessing techniques and training-validation-test splitting protocols as previously introduced in [12] for our evaluation. Specific details are summarized in Table I.

Implementational details: Without loss of generality, we implement dynamic early-exit on top of the recently proposed state-of-the-art convolutional network architecture [9], [10] to demonstrate that our approach can effectively improve the accuracy-efficiency trade-offs of such compute-intensive deep models. Unless otherwise specified, we employ a standard six-layer CNN as our backbone while modifying it with five exits, where the channel numbers in these layers are 8, 16, 32, 64, 128, and 256 respectively, and the kernel size is uniformly set to (3,1). Except the first layer, these intermediate classifiers are inserted after each convolutional layer, which comprise a regular convolutional layer with the same kernel size, followed by an average-pooling layer, and a full-connected layer. To evaluate the backbone extensibility of our approach, we also design a simple residual backbone network comprised of five residual blocks and then deploy our sample-reweighting approach on it with four exits, where these intermediate classifiers are inserted after

TABLE I
STATISTICAL INFORMATION OF DATASETS

Datasets	UCI-HAR	UniMiB-SHAR	WISDM
Subjects	36	30	30
Activity classed	Standing, Sitting, Lying down, Walking, Descending stairs, Ascending stairs	Sitting down, Running, Walking, Falling backward, Jumping, Syncope, etc.	Stair climbing, Sitting, Standing, Walking, Jogging
The number of activity classes	6	17	6
Window length	200	151	128
The number of sensor modalities	3	3	9
Sampling rate	20Hz	50Hz	50Hz
Sensing device	Nexus One, HTC hero, Motorola Backflip	Samsung Galaxy Nexus I9250	Samsung Galaxy S II
Sensor location	Front leg pocket	Front trouser pocket	Left side of the belt
Sensor modalities	Accelerometer	Accelerometer	Accelerometer, Gyroscope
Train set	7686	7734	7352
Validation set	1099	1346	983
Test set	2196	2691	1964

each residual block, except the first residual block. To maintain lightweight property, we restrict the number of layers in each residual block to 1 while adding skip connection. Moreover, our proposed approach is compared with numerous highly competitive dynamic or compression baselines, i.e., MSDNet [24], DenseNet [27], Global Pruning [28], ABCPruner [29]. At an initial learning rate of 1×10^{-3} , we employ an Adam optimizer with a weight decay of 1×10^{-3} to train the backbone weights Θ_f and our WPN Θ_g through 300 epochs (with or without our meta-learning based mechanism), where the batch sizes are uniformly set to 64 for all three HAR datasets. While applying our sample-reweighting algorithm, one mini-batch must be divided into two equal parts, among which one half is responsible for updating the backbone weights (i.e., Θ_f), while the other half is responsible for updating the WPN weights (i.e., Θ_g). In such a way, one is able to alternately exchange both of them during each iteration, which ensures that the number of total training iterations may be equivalent to the other conventional comparing baselines without applying the meta-learning based mechanism. Based on the hold-out validation set, we empirically explore different values for the budget control variable q [24] and the perturbation magnitude δ (i.e., the weighted classification loss as introduced in Section III-B) to determine the best hyperparameter-selection. Specifically, we find that a q value tuned at 0.5 can work well for both UCI-HAR and UniMib-SHAR, while a q value of 0.375 is optimal for WISDM. With regards to δ , we set it to 0.5 for UCI-HAR, and 0.3 for other two datasets (i.e., UniMiB-SHAR and WISDM). We released the related codes about specific structural designs and training details at: <https://github.com/FuZenan/LtSRW-for-EEAR> to aid better understanding and replication of the results.

B. Main Results

Following [24], we evaluate our meta-learning based activity recognition algorithm under two scenarios: (1) Budgeted batch inference, the network requires to classify a set of activity samples within a restricted computational budget, where we must estimate the confidence thresholds to execute adaptive activity inference; (2) Anytime prediction, the network will be

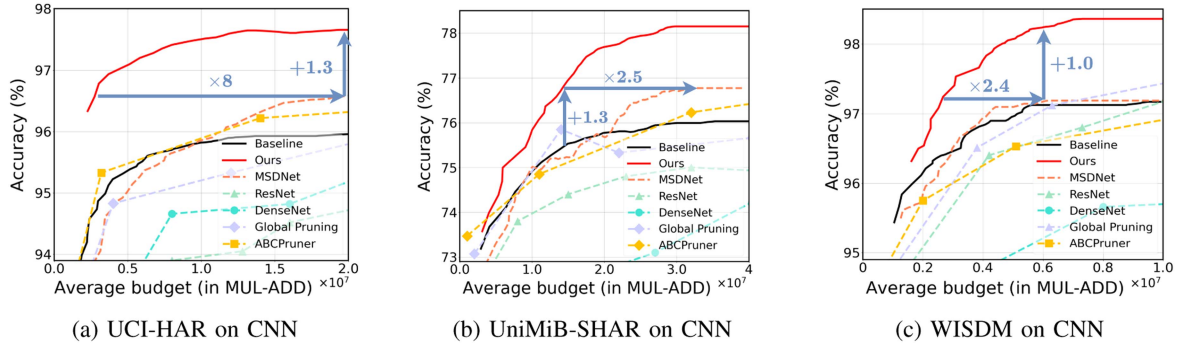


Fig. 3. Budgeted batch inference results on UCI-HAR, UniMiB-SHAR, and WISDM datasets using CNN.

TABLE II
ANYTIME RESULTS OF 5-EXIT CNN

Exit index			1	2	3	4	5
Params (M)			0.02	0.05	0.12	0.31	0.82
UCI	Mul-Add (M)		0.52	2.38	9.63	38.29	152.24
	Baseline	Accuracy	93.63%	94.86%	95.11%	95.47%	95.89%
		F1-score	93.47%	94.85%	95.09%	95.46%	95.89%
		Recall	93.61%	94.86%	95.11%	95.47%	95.89%
	Ours	Accuracy	94.50%	95.88%	96.79%	97.15%	97.66%
		Δ Accuracy	↑0.87%	↑1.03%	↑1.70%	↑1.69%	↑1.77%
		F1-score	94.48%	95.87%	96.79%	97.15%	97.66%
		Δ F1-score	↑1.01%	↑1.03%	↑1.70%	↑1.68%	↑1.77%
		Recall	94.50%	95.88%	96.79%	97.14%	97.66%
	Δ Recall	↑0.99%	↑1.03%	↑1.68%	↑1.66%	↑1.77%	
UniMiB	Mul-Add (M)		0.23	1.01	3.97	15.46	60.70
	Baseline	Accuracy	69.64%	72.58%	74.73%	75.29%	76.03%
		F1-score	69.31%	72.00%	74.41%	74.95%	75.61%
		Recall	69.61%	72.58%	74.73%	75.29%	75.99%
	Ours	Accuracy	70.22%	73.84%	76.11%	77.15%	78.15%
		Δ Accuracy	↑0.58%	↑1.26%	↑1.38%	↑1.86%	↑2.12%
		F1-score	70.11%	73.39%	75.48%	76.92%	77.99%
		Δ F1-score	↑0.80%	↑1.39%	↑1.07%	↑1.97%	↑2.38%
		Recall	70.22%	73.84%	76.11%	77.15%	78.15%
	Δ Recall	↑0.61%	↑1.26%	↑1.38%	↑1.86%	↑2.16%	
WISDM	Mul-Add (M)		0.30	1.33	5.24	20.42	80.27
	Baseline	Accuracy	94.02%	95.71%	96.43%	96.81%	97.21%
		F1-score	93.87%	95.64%	96.44%	96.80%	97.23%
		Recall	94.03%	95.71%	96.43%	96.81%	97.21%
	Ours	Accuracy	94.63%	96.86%	97.22%	98.08%	98.36%
		Δ Accuracy	↑0.61%	↑1.15%	↑0.79%	↑1.27%	↑1.15%
		F1-score	94.31%	96.85%	97.20%	98.10%	98.36%
		Δ F1-score	↑0.44%	↑1.21%	↑0.76%	↑1.29%	↑1.13%
		Recall	94.40%	96.86%	97.22%	98.09%	98.36%
	Δ Recall	↑0.37%	↑1.15%	↑0.79%	↑1.28%	↑1.115%	

forced to return an activity prediction at an arbitrary given time point, which assumes that the early-exit point is always the same for all activity samples. As introduced in [24], both scenarios are ubiquitous in real-world HAR deployment. Fig. 3 and Table II respectively report the main results obtained through representative convolutional backbone architecture on multiple HAR benchmarks, i.e., UCI-HAR, WISDM and UniMiB-SHAR. Without loss of generality, the proposed sample-reweighting method is compared with three types of competitive baselines:

classic networks (ResNet [30], DenseNet [27]), compression-related methods (Global filter pruning [28], Structural Pruning (ABCPruner) [29]), and other state-of-the-art early-exit architecture (MSDNet [24]).

Budgeted batch inference results: Under such dynamic inference scenario, the computational budget can be precisely controlled in advance by predefining a handful of confidence thresholds on multi-exits, which enable the model to allocate different computing resources for various activity samples based on their input complexity. As illustrated in Fig. 3, we plot the accuracy-computation curves measured in terms of Mul-Adds (left: UCI-HAR, middle: UniMiB-SHAR, right: WISDM). This figure indicates that the introduced sample-reweighting approach is able to significantly boost the performance of multi-exits networks across all computational budget levels. While applied to the 5-exit network, our sample-reweighting strategy yields considerable performance boost, and obtains a recognition accuracy of 97.7% on UCI-HAR dataset, which outperforms the vanilla baseline by $\sim 1.8\%$ when evoking around 8M Mul-Adds, with an accuracy gain of $\sim 2.1\%$ on UniMiB-SHAR dataset while evoking around 40M Mul-Adds. Our model achieves the highest accuracy of $\sim 98.36\%$ with minimal computational resources on WISDM dataset, compared to other baselines. We also observe that the proposed method may be significantly superior to other efficient state-of-the-art comparing baselines while consuming almost the same amount of computation, which surpasses the second best baseline such as MSDNet by about 1.3% and significantly improves the accuracy of static models (ResNet / DenseNet) by 2.2% $\sim$ 3.2% on UCI-HAR dataset. On the other hand, while keeping almost the same accuracy, it is able to achieve a considerable speedup. For example, our proposed method is shown to consume $\sim 8.0\times$ less computation compared with MSDNet on UCI-HAR dataset, and $\sim 2.5\times$ on WISDM dataset. Moreover, one can expect that our meta-learning based early-exit method will be well compatible with these strong backbone networks, e.g., residual backbone can also be trained with our sample-reweighting strategy (as shown in later Fig. 4) to further boost the accuracy and reduce computational budgets. With an average budget of 4M Multiply-Adds, the modified early-exit ResNet model integrating with our WPN module achieves a superior validation accuracy of 97.1%, which outperforms the vanilla static ResNet by $\sim 4.1\%$. Notably, comparing to these static baselines, our method provides a precise control over accuracy-cost trade-offs, which can flexibly attain all the points on every budget curve by simply adjusting the values of confidence threshold on multi-exits [24], offering adaptiveness for activity inference on edge devices.

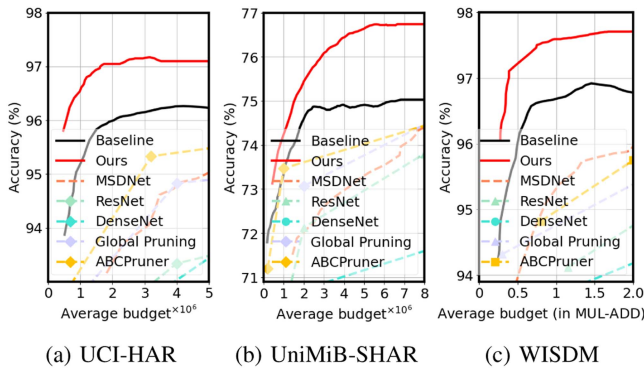


Fig. 4. Budgeted batch inference results on UCI-HAR, UniMiB-SHAR, and WISDM datasets using ResNet.

Anytime prediction results: In addition to budgeted batch inference results, we summarize the relevant accuracy of every exit in anytime prediction mode, where all activity samples are forced to exit from the same classifier while returning its most up-to-date prediction within an arbitrary wanted time regardless of their hardness levels, so as to meet strict limitations on inference delay. From results in Table II, it can be observed that our method outperforms the conventional early-exit baselines by large margins, improving the accuracy at each exit for each dataset by approximately 1.0% ~ 2.0%. This demonstrates the effectiveness of the designed sample-reweighting mechanism. While our meta-based optimization objective typically encourages every exit to only attend to a subset of its most confident activity samples, this would be beneficial for improving overall performance across various HAR tasks. Shallow exits highlight a smaller set of easily recognized samples with high confidence, while deeper exits focus on challenging activity samples that previous classifiers could not confidently predict. This aligns well with our later ablation study findings (refer to the middle panel in Fig. 7), indicating that promoting each exit to preferentially emphasize a subset of carefully chosen training samples would be meaningful. Additionally, we comprehensively evaluate the performance of our model across all datasets through two common metrics: F1 Score and Recall, among which the F1 Score accounts for class imbalance that can be viewed as a measure of the harmonic mean of Precision and Recall. As shown in Table II, our model achieves consistent performance improvements in these metrics, indicating that it is very reliable and effective. Particularly, we can clearly observe higher F1 score and Recall with relative gains of 2.1% and 2.4% respectively on the imbalanced UniMiB-SHAR dataset, where unlike common daily activities these falls actions are relatively hard to annotate and collect. This demonstrates that our method can naturally maintain a substantial advantage in handling class imbalance HAR problem.

ResNet results: In order to evaluate the generality ability of the designed sample-reweighting approach, we perform additional comparison experiments on the other strong backbone architecture, i.e., ResNet. Fig. 4 summarizes the budgeted batch inference results of a 4-exit ResNet on the three used HAR benchmarks. The experimental results show that our proposed approach is able to consistently improve accuracy-computation trade-offs. Notably, when the multiply-add operations (Mul-Add) are approximately 5 M, our model significantly boosts the accuracy by around 1.9%, compared to conventional early-exit training scheme. The consistent performance gains across all

three datasets verify the generality and effectiveness of our sample-reweighting algorithm.

Confusion Matrices: To clearly validate the effectiveness of the proposed approach, we compare it with the conventional early-exit baseline model without applying our meta-learning idea, by predefining three different confidence thresholds to calculate the confusion matrices on the UCI-HAR dataset under different computational budgets. As illustrated in Fig. 5, we can see a noticeable misclassification between “Sitting” and “Standing” as highlighted by red rectangles, which is due to their similar signal waveforms caused by static behavior. For example, under low computational budgets, there are 54 “Sitting” activity samples that have already been wrongly recognized as “Standing”. While further increasing the computational budget, the performance of the conventional early-exit baseline model could not reach an ideal level. In contrast, through adaptively imposing appropriate weights on the training loss of different activity samples, our meta-learning based strategy is able to considerably decrease the number of misclassifications to a much lower level, as shown by Fig. 5.

Training-time complexity analysis: In fact, incorporating such meta-learning into dynamic early-exit networks would incur additional training-time complexity, due to the need of jointly optimizing both backbone network and weight-predicting network (WPN) during training phase. As illustrated in Fig. 6, we present the training errors averaged over all exits on UCI-HAR dataset, as the training time evolves. In comparison with standard training, it can be seen that integrating meta learning with dynamic early-exit network does increase training-time complexity, which significantly slows down the training speed, requiring more training time to converge. However, our method can reach a lower plateau. In industry, the researchers or engineers often have abundant training resources but strict limitations on the inference-time costs. While decoupling training-time and inference-time costs, it is reasonable to train deep models on powerful advanced GPU workstations and then deploy the trained HAR models to resource-constrained or latency-sensitive devices. Considering this, we usually judge the quality of deep model by the trade-off between classification accuracy and inference-time costs (i.e., latency) in the ubiquitous HAR computing scenario. Therefore, in addition to the benefits of adaptiveness or flexibility, we consider it acceptable to accelerate activity inference at the cost of more training-time resources, as long as the final deployed inference-time model has lower inference-time cost, while preserving satisfactory accuracy.

C. Ablation Studies

The impact of meta-learning algorithm: To validate the necessity of such meta-learning mechanism, in addition to previous vanilla baseline where all weights are uniformly fixed to 1, we compare it with two other main variants: The former variant is manual design that replaces the sample-reweighting mechanism in our implementation with fixed weights, which are monotonically decreased from 1.4 to 0.6 (or increased in reverse order) with uniform steps as increasing exit index; The latter variant is to apply a frozen WPN already converged for sample-reweighting. Results on UCI-HAR are shown in Fig. 7 (i.e., the left subplot), from which we can find that our proposed approach preforms better consistently than the vanilla baseline with three other variants, demonstrating the significance of the sample-reweighting mechanism. The accuracy gain brought by the weight values manually predefined is rather

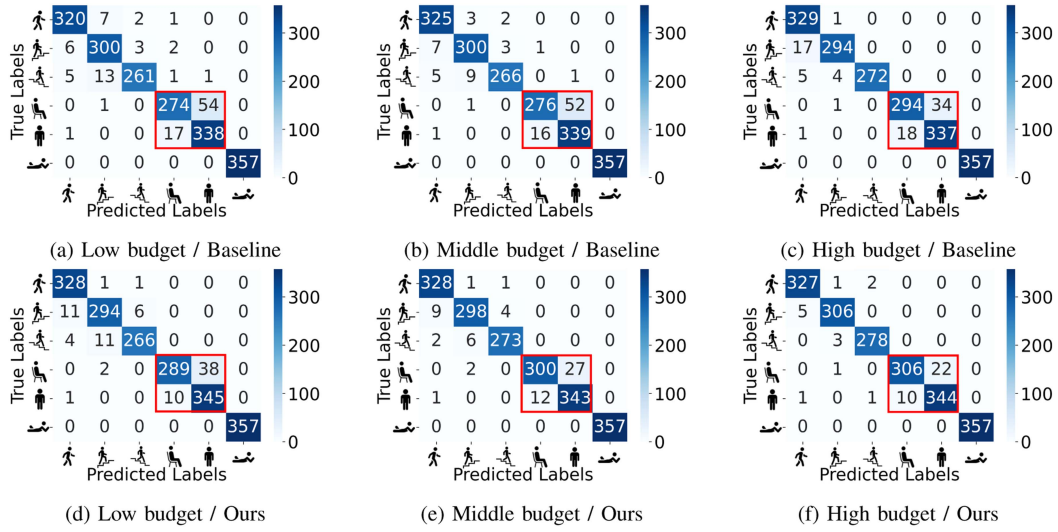


Fig. 5. Confusion Matrices on UCI-HAR dataset under different budgets.

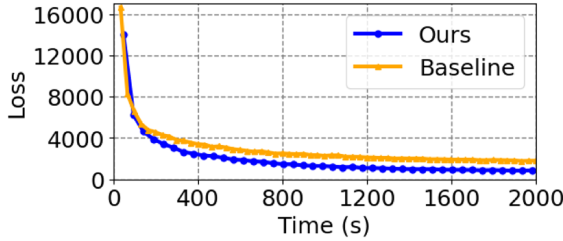


Fig. 6. Training loss curves.

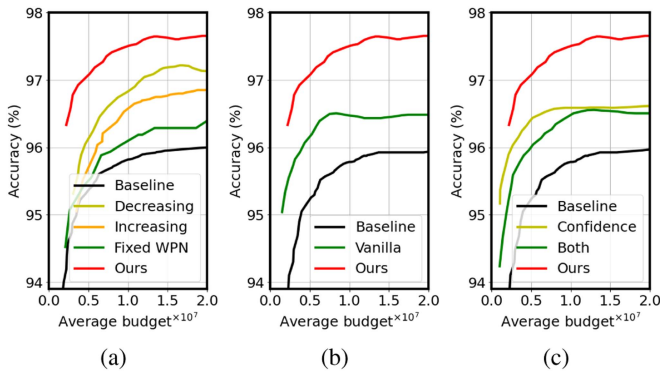


Fig. 7. Budgeted batch inference results on UCI-HAR(Top). Left: Comparison of our meta-learning algorithm with different weighting mechanisms. Middle: The effectiveness of our meta objective. Right: Comparison of different WPN inputs.

limited, suggesting the superiority of our sample-reweighting mechanism. Among them, our jointly optimized model performs the best, which significantly surpasses its counterpart with sample-reweighting through the frozen WPN, indicating that it is crucial to learn the appropriate weights adaptively imposed on the different activity samples during training phase.

The effect of meta-learning objective: As introduced in (6), our designed meta objective is able to encourage each exit to precisely identify a subset of its most confident activity samples, which have not been output at previous exits, but are highly

likely to be handled by current exit, thereby introducing the test-time dynamic behavior into the training stage. To verify the necessity of our meta objective, while remaining the whole learning process unchanged, we only replace (6) with the classification loss applied over the entire meta set for every exit. Fig. 7 summarizes the results evaluated on UCI-HAR. Though this substituting solution (referred to as Vanilla Objective in the middle panel) clearly surpasses our vanilla baseline, our proposed meta-learning objective is more effective and can consistently achieve higher accuracy at all computational budget levels, especially at larger computational budgets, which again indicates that it is essential to emphasize various training samples at multiple exits with different depths for performance improvement.

The input of WPN: Besides the classification loss, since the confidence value has always played a crucial role in making such early-exit decisions, it could be utilized to generate weight perturbations as well. Therefore, we further check three different types of input for WPN: 1) the loss alone; 2) the confidence value alone; 3) the combination of loss and confidence. Fig. 6 plots the accuracy-cost curves on UCI-HAR (as see the right panel in the figure). It can be seen that case 1 significantly outperforms other two cases, which indicates that it is essential to only apply loss in our implementation, while including the confidence value would be somewhat detrimental for model performance. Thus, it hypothesized that only the loss function itself is already able to provide an enough good candidate as WPN input by integrating the information from both network predictions and the ground truth.

Class imbalance results: In fact, different from image data, it is rather laborious and time-consuming to collect and annotate intricate sensor data. For example, some unexpected or emergent activities such as accidental fall is especially difficult to acquire, which cause a challenging problem called class imbalance for sensor-based HAR. Therefore, our meta-learning based approach is also evaluated under such class imbalance setting. To this end, by randomly choosing or removing a certain number of activity samples on the UCI-HAR dataset, we test the proposed with three different imbalance factors (20, 10, 5), which can be denoted as the number of training samples in the largest class divided by the smallest. To be specific, to produce a

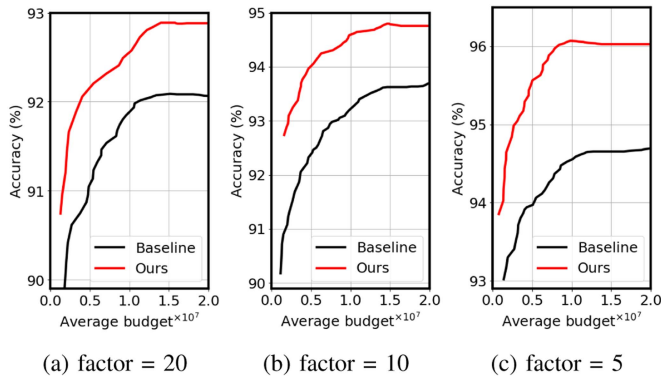


Fig. 8. Results on UCI with different imbalance factors.

class-imbalance distribution, we decrease the number of training samples within each activity class according to an exponential function $N'_c = N_c \mu^c$, $\mu \in (0, 1)$, in which c represents the class index, while N_c denotes the number of original training samples. Fig. 8 present the results under budgeted batch inference mode, which show that the proposed sample-reweighting method can consistently surpass the comparing baseline trained by the traditional training scheme by a large margin, indicating the significance of sample-reweighting under class imbalance HAR setting.

Ablation on different depths: We further conduct an ablation study to validate the effectiveness of our proposed approach across different network depths. Apart from varying network depths, all of the tested methods still apply the similar backbone architectures on the UCI-HAR dataset introduced in previous main results (Section IV-A). By preserving the original structure of each layer, we compare our meta-learning based method with the vanilla baselines constructed at four different depths (3, 4, 5, 6) by inserting the intermediate early-exit classifier after each layer except the first layer, resulting in total 3, 4, 5, and 6 exit points respectively. Results under budgeted classification mode illustrated in Fig. 9 show that the proposed approach can consistently improve the performance of such dynamic multi-exit network at different depths, no matter whether it's deep or shallow.

Visualization: To better understand the proposed approach, Fig. 10 visualizes the predictive weights by WPN trained on UCI-HAR, where the left subfigure reveals the weight-loss relationship from the first and third exits respectively, while the right one illustrates those related weights of activity samples chosen or unchosen at the first exit through such data allocation based on meta learning, as previously introduced in Section III-B. From the visualized results, we can observe three prominent cases: First, during training, while fixing q at 0.5, we could roughly achieve an exponential distribution of activity samples outputted at five exits with a proportion of [0.52, 0.26, 0.13, 0.06, 0.03]. On average, since more samples are allocated to earlier exits, they should bear more responsibility for those activity samples and hence generally assign higher weights to them at exit-1 rather than at exit-3, suggesting that the proposed approach can successfully establish the weight-loss relationship across different exits; Second, there is an obvious concave downward trend along two weight-loss relationship curves, which indicates that both of exits tend to underline more activity samples at a relatively small loss, agreeing well with previous observation

in [31]: In practice, most well-predicted samples with higher prediction confidences should contribute smaller loss to gradient back-propagation, hence potentially hindering representational learning. To mitigate this deficiency, our approach can revive their contribution to such learning procedure by rewarding well-classified activity examples (i.e., easy) with sample-reweighting. Third, according to the averaged weights from exit-1, it can be perceived that the activity samples with higher prediction likelihoods (i.e., that would be chosen by the first exit through meta-based data allocation) can preserve totally larger weights compared to those with lower prediction likelihoods (that would not be selected at the first exit and further allocated to deeper or later exits), which shows that the proposed meta-based approach can well learn where to an activity sample will exit at different classifiers.

Backbone extensibility: We further verify the backbone extensibility of our proposed approach on other deep learning structures. To achieve this goal, we apply our sample reweighting strategy to LSTM and DeepConvLSTM with five exits, and then compare it with the standard early-exit training technique on UCI-HAR dataset. In the former case, there are five LSTM layers with 8, 16, 32, 64, and 128 LSTM units within them, followed by a softmax and a standard classification layer. In the latter case, DeepConvLSTM is a popular hybrid framework [32] comprised of four convolutional layers and two LSTM recurrent layers, where these intermediate classifiers are inserted after each convolutional block for early-exit activity recognition. Under budgeted batch inference pattern, we plot the accuracy-computation (measured by Mul-Adds) curves in Fig. 11 (left: LSTM, right: DeepConvLSTM). From the results, it can be seen that our proposed sample-reweighting approach can consistently improve the performance of two backbones across various computational budgets. To be specific, while applied to the 5-exit LSTM model, our sample-reweighting mechanism can achieve significant boosts with the accuracy increased by round 2.5% when evoking about 15M Mul-Adds. For the 5-exit DeepConvLSTM, our method achieves an about 1.7 $\times$ speedup while obtaining round 93.0% accuracy on UCI-HAR dataset.

D. Hardware Deployment

Since theoretical FLOPs could not directly translate into practical inference delay, we further analyze the practical activity inference speed of the proposed approach on a specific hardware device. It is well known that the low-cost Raspberry Pi devices have a good compatibility with current mainstream PyTorch deep learning toolkit. For sake of convenience, we directly select Raspberry Pi 4 Model B as our target platform for performance evaluation. We deploy the meta-learning based dynamic network on target platform, which involves three main steps: The first step is to train our designed model with optimal performance on the UCI-HAR training set; The second step is to load the saved model (e.g., .pth file) into the target platform, and then call `model.eval()` to set the BN layers to evaluation mode before performing forward-pass inference; At the third step, a 'time' library is utilized to measure inference time. While the loaded model starts to read an activity example, a timer will be launched (denoted as 'Starting_time'), and then be stopped once a confident prediction is returned (denoted as 'ending_time'). A simple subtraction operation can be performed to estimate the time elapsed between them during forward-pass inference. In addition, we configure the budgeted classification setting

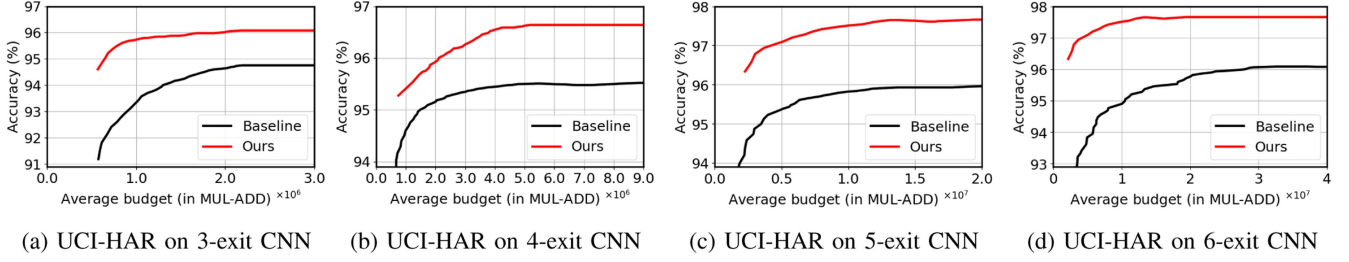


Fig. 9. Compare of different depth of the model.

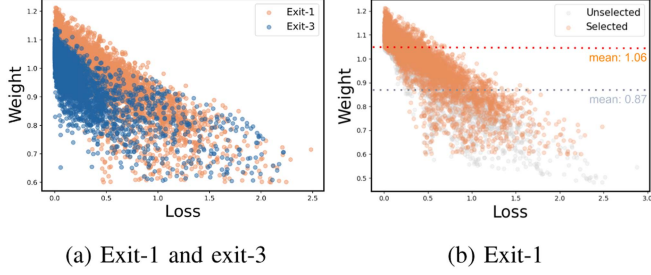


Fig. 10. Visualization results of the weight-loss relationship on UCI-HAR.

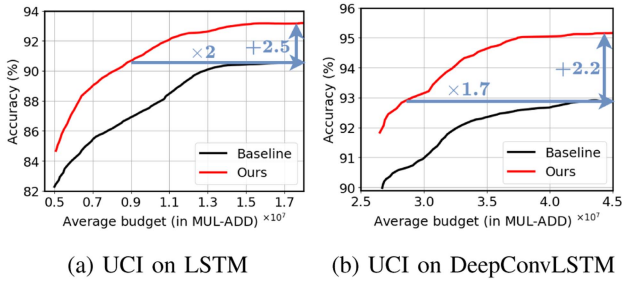


Fig. 11. Budgeted batch inference results on UCI-HAR dataset using LSTM and DeepConvLSTM backbone.

under a single-thread mode. As previously introduced, under such budgeted inference pattern, some ‘easy’ activity samples that can be confidently predicted (i.e., surpassing the predefined thresholds) will exit this forward-pass procedure in advance, while the remaining activity samples will be further processed through later exits, until they are able to meet the early-exit condition or reach the final classifier. We develop a HAR application program (written in Python language) to offer an exemplar illustration, whose main user interface can be seen in Fig. 12. We measure the practical inference times of total 1964 test activity samples under low, medium, and high budget levels by pre-defining three different confidence thresholds: [0.52, 0.26, 0.13, 0.06, 0.03], [0.20, 0.20, 0.20, 0.20, 0.20], and [0.03, 0.06, 0.13, 0.26, 0.52], as illustrated from top to bottom in Fig. 13. We also present a detailed sample distribution of different activity categories at each exit through pie chart. It can be seen that the average inference delays are 3.47 ms, 7.58 ms, and 11.28 ms under the three computational budgets, respectively. Notably, compared to the static CNN having almost the same model size, our meta-learning method is able to considerably accelerate the activity inference by round $4.72 \times$ (3.47 ms vs. 16.37 ms), $2.16 \times$ (7.58 ms vs. 16.37 ms), and $1.45 \times$ (11.28 ms vs. 16.37 ms)

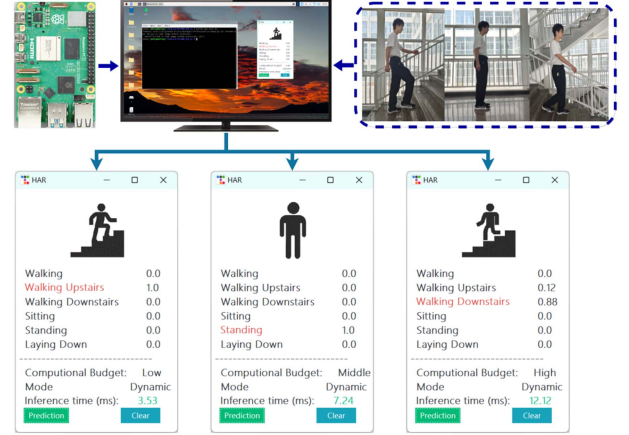


Fig. 12. Graphical User Interface of a HAR Application on Raspberry Pi with varying computational budgets.

while preserving higher accuracy, indicating its practicality and usefulness in real-world HAR scenario.

Under the same setting, we further present the trade-offs of energy consumption and accuracy in our early-exit neural network using Raspberry Pi and Nvidia Jetson nano devices, as illustrated in Fig. 14. Since it is often difficult to directly measure the energy consumption of various devices, we instead estimate it based on the time elapsed while our program consumes a fixed power. The specific measurement formula is as following:

$$P = \frac{Q \cdot V \cdot 3600}{T}, \quad (10)$$

where T denotes the time (s) while our program consumes a fixed amount of power. Given that the rated power we used was $Q = 20000$ mAh and the rated voltage of the devices was 5 V, we measured the actual time elapsed while consuming round 1% power. Substituting them into (10), we can get: $P = \frac{3600}{T}$. As shown in Fig. 15, in the case of Raspberry Pi, our model takes 405.68 s, 337.14 s, and 313.35 s to consume 1% power under low, medium, and high budgets respectively, while the static backbone baseline takes 245.56 s. According to (10), the corresponding energy consumptions of our early-exit network are 8.87 W, 10.68 W, and 11.48 W respectively, while that of the static backbone is 14.66 W. In the case of Nvidia Jetson nano, our model takes 343.02 s, 304.45 s and 278.42 s to consume 1% power under low, medium and high budgets respectively, while the static backbone baseline takes 224.16 s. According to (10), our energy consumptions are 10.49 W, 11.82 W and 12.93 W under low, medium and high budgets respectively, while that of the static backbone baseline is 16.06 W. The results indicate that

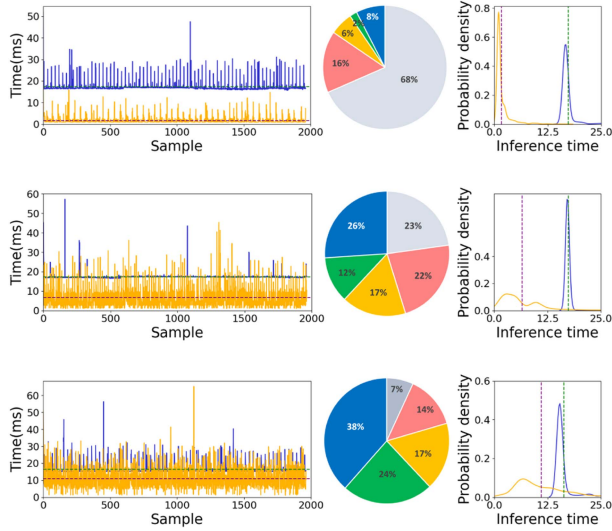


Fig. 13. Inference time statistics under varying computational budgets. Top: Low budget. Middle: Middle budget. Bottom budget: High budget. Different colors in the pie chart represent the proportion of exit samples from our model at different budget levels. Gray: Exit-1, Red: Exit-2, yellow: Exit-3, Green: Exit-4, Blue: Exit-5; The line in our picture with different colors also have different meanings, which the orange line and the dark blue line represent dynamic inference time and static inference time respectively, the light green line and the purple line represent average static inference time and average dynamic inference time respectively.



Fig. 14. Deployment diagram of energy consumption measurement.

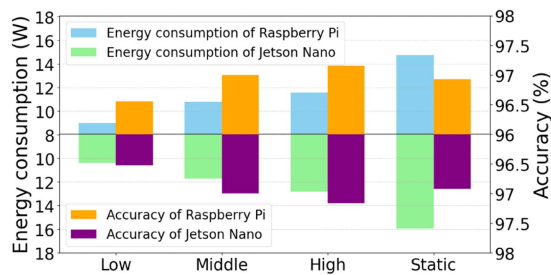


Fig. 15. Energy consumption vs. accuracy trade-offs.

our method can also tradeoff energy consumption and accuracy well, while deployed on different hardware platforms.

E. Cross-Domain Evaluations

In fact, there have recently been only a few works [33], [34] that focus on bridging the domain gap in the early-exit pretrained language models (PLMs) from a cross-domain perspective. So far, we have not seen related literatures reported in the context of HAR. We further perform comprehensive evaluations by training our convolutional backbone with four exits on a source domain dataset and assessing its performance and speedup on

TABLE III
ACCURACY(%) V.S. SPEEDUP TRADEOFFS IN CROSS-DOMIAN SETTING

	Target Domain	Static ERM	Static LISA	Early-exit Baseline	Early-exit LISA	Ours	Ours LISA
Cross-person	0	82.5	92.2	72.7	82.3	75.4	90.8
	1	78.4	89.8	73.0	89.7	77.9	89.9
	2	82.1	91.5	83.0	87.0	88.6	88.1
	3	92.8	95.1	90.3	91.7	84.7	90.5
	4	90.2	92.4	86.0	89.8	83.9	92.2
	AVG	85.2	92.6	80.8	88.1	82.1	90.3
	SPEEDUP	1.00x		1.90x		2.15x	
Cross-dataset	0	43.7	45.4	33.1	41.2	30.3	42.3
	1	26.2	30.1	22.5	29.3	21.1	26.6
	2	28.3	28.5	24.2	29.0	24.3	27.6
	3	32.0	34.2	26.2	29.8	25.7	28.1
	AVG	32.6	34.6	26.5	32.3	25.4	31.2
	SPEEDUP	1.00x		1.64x		2.4x	

a target domain dataset that exhibits distribution variation from the source dataset. Following the previous literature [35], we select the first 5 subjects' data (ID: 0, 1, 2, 3, and 4) from the UCI-HAR dataset to form such cross-person setting, where their activity patterns vary from subject to subject, but still shares the same activity type. To the best of our knowledge, this is the first time to perform cross-domain evaluations in an early-exit HAR scenario. To this end, we compare our method against both backbone and standard early-exit baseline, with/without applying domain generalization (DG). However, intuitively, it is not easy to improve generalization of an early-exit model by most of existing DG techniques, as they only rely on the prediction of the last layer, which is in contradiction to multiple layers involved in early-exit model. To avoid this issue, we exploit a model-agnostic mixup-based DG technique called LISA [36] for our cross-domain evaluation, which may perform data augmentation by interpolating sensor samples either with the same domain but different labels or with the same labels but different domains. Table III summarize the results in terms of accuracy and speedup. Without performing DG technique, it can be seen that the performance of both ours and standard early-exit baseline suffers from overfitting on the source domain, due to poor generalization. Though our method surpasses the standard early-exit baseline, it still significantly underperforms the backbone baseline. While applying LISA, we find that it can effectively improve our method by data augmentation, which does achieve comparable generalization performance, as well as better speedup. The results indicate that our method can serve as an acceptable foundation model to achieve an ideal trade-off of generalization performance and speedup.

Furthermore, in addition to the cross-person scenario, we conduct another more challenging cross-domain experiment (namely cross-dataset) to check the generalizability of our approach. Following the same setting in previous literature [37], we merge four public HAR datasets including UCI-HAR [6], DSADS [38], USC-HAD [39], and PAMAP2 [40] collected from various environments or different sensor modalities, where every dataset may be viewed as a different domain. Such cross-dataset scenario is more challenging than cross-person one, because the involved sensor positions, devices, subjects are various. From the results in Table III, we can see that the backbone baseline, the standard early-exit baseline, and our method experience a dramatic performance degradation. After performing DG with LISA, we find that it can consistently improve all three baselines.

Though our method is inferior to the backbone baseline, it is worth noting that there is still huge room for further performance improvement in such cross-dataset scenario. To simultaneously get the best of both robustness performance and speedup, we hypothesize that one promising solution (as motivated in [33]) is to learn domain-invariant representations across all the layers such that every exit classifier trained on a source domain can be directly exploited for the target domain, which is another long story. We will leave this as our future work.

V. CONCLUSION

This paper introduces a new method based on meta-learning mechanism to train dynamic early-exit networks, which can adaptively adjust sensor sample weights for efficient activity inference. Our main goal is to incorporate test-time adaptive behavior during training by assigning different weights to individual activity sample at each exit, based on its practical difficulty level. To achieve this, we employ a novel meta-learning objective to optimize a weight-predicting network alongside the backbone network. The experimental results demonstrate it is able to consistently improve the performance of dynamic multi-exit networks under anytime prediction scenario as well as budgeted batch classification mode, which verify the effectiveness and efficiency of our proposed approach. Moreover, our sample-reweighting approach based on meta-learning mechanism naturally maintain a substantial advantage in tackling class imbalance HAR problem. Detailed ablation studies and intuitive visualizing illustrations are provided to support our statement. Our work is highly relevant to real-world Human Activity Recognition (HAR) scenarios implemented on wearable devices, leading to significant reductions in resource consumption and inference delay. An actual evaluation is also performed on a mobile device, indicating its practicability and usefulness.

Potential limitation and future work: While our main study focus is early-exit activity inference, another interesting concern is its effectiveness and efficiency in a cross-domain setting. In fact, most existing early-exit HAR works typically assume that every activity sample must be inferenced an intermediate layer or the final layer, while its obtained confidence is above a certain threshold. In such setup, one has to empirically pre-determine the threshold by utilizing the validation set during training [24], which may serve as a major reference point for multi-exit inference. However, it poses a significant challenge while deploying a pre-trained model that is then tested on activity samples whose underlying distribution might differ from the training samples under cross-domain setting. Since the distribution of predicted confidences from the attached intermediate classifiers may vary while transitioning across domains, this would render our early-exit model less efficient, given that the thresholds could not be adapted as per the underlying distribution of new unseen samples. It still remains a main challenge in real-world: How to adaptively the threshold while deploying such early-exit model to maintain its efficiency and robustness to domain-shift in a cross-domain setting? This has been rarely explored in post-deployment HAR scenario, which will be left as our future work.

REFERENCES

- [1] H. Wang, P. Kang, Q. Gao, S. Jiang, and P. B. Shull, "A novel PPG-FMG-ACC wristband for hand gesture recognition," *IEEE J. Biomed. Health Inform.*, vol. 26, no. 10, pp. 5097–5108, Oct. 2022.
- [2] K. Chen, D. Zhang, L. Yao, B. Guo, Z. Yu, and Y. Liu, "Deep learning for sensor-based human activity recognition: Overview, challenges, and opportunities," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 1–40, May 2021.
- [3] R. Mishra and H. Gupta, "Transforming large-size to lightweight deep neural networks for IoT applications," *ACM Comput. Surv.*, vol. 55, no. 11, pp. 1–35, 2023.
- [4] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7436–7456, Nov. 2022.
- [5] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "SkipNet: Learning dynamic routing in convolutional networks," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 409–424.
- [6] D. Anguita et al., "A public domain dataset for human activity recognition using smartphones," *Sensors*, vol. 3, 2013, Art. no. 3.
- [7] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, "Activity recognition using cell phone accelerometers," *ACM SigKDD Explorations Newslett.*, vol. 12, no. 2, pp. 74–82, 2011.
- [8] D. Micucci, M. Mobilio, and P. Napoletano, "UniMiB SHAR: A dataset for human activity recognition using acceleration data from smartphones," *Appl. Sci.*, vol. 7, no. 10, 2017, Art. no. 1101.
- [9] J. B. Yang, M. N. Nguyen, P. P. San, X. L. Li, and S. Krishnaswamy, "Deep convolutional neural networks on multichannel time series for human activity recognition," in *Proc. 24th Int. Conf. Artif. Intell.*, 2015, pp. 3995–4001.
- [10] M. Zeng et al., "Convolutional neural networks for human activity recognition using mobile sensors," in *Proc. IEEE 6th Int. Conf. Mobile Comput. Appl. Serv.*, 2014, pp. 197–205.
- [11] C. A. Ronao and S.-B. Cho, "Human activity recognition with smartphone sensors using deep learning neural networks," *Expert Syst. Appl.*, vol. 59, pp. 235–244, 2016.
- [12] N. Y. Hammerla, S. Halloran, and T. Plötz, "Deep, convolutional, and recurrent models for human activity recognition using wearables," in *Proc. 25th Int. Joint Conf. Artif. Intell.*, 2016, pp. 1533–1540.
- [13] W. Jiang and Z. Yin, "Human activity recognition using wearable sensors by deep convolutional neural networks," in *Proc. 23rd ACM Int. Conf. Multimedia*, New York, NY, USA: Association for Computing Machinery, 2015, pp. 1307–1310.
- [14] S. Münzner, P. Schmidt, A. Reiss, M. Hanselmann, R. Stiefelhausen, and R. Dürichen, "CNN-based sensor fusion techniques for multimodal human activity recognition," in *Proc. ACM Int. Symp. Wearable Comput.*, New York, NY, USA: Association for Computing Machinery, 2017, pp. 158–165.
- [15] M. A. A. Al-qaness, A. Dahou, M. A. Elaziz, and A. M. Helmi, "Multi-ResAtt: Multilevel residual network with attention for human activity recognition using wearable sensors," *IEEE Trans. Ind. Inform.*, vol. 19, no. 1, pp. 144–152, Jan. 2023.
- [16] F. Luo, S. Khan, Y. Huang, and K. Wu, "Binarized neural network for edge intelligence of sensor-based human activity recognition," *IEEE Trans. Mobile Comput.*, vol. 22, no. 3, pp. 1356–1368, Mar. 2023.
- [17] J. Liang, L. Zhang, C. Han, C. Bu, H. Wu, and A. Song, "A collaborative compression scheme for fast activity recognition on mobile devices via global compression ratio decision," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3259–3273, Apr. 2024.
- [18] Z. Chen, L. Zhang, Z. Cao, and J. Guo, "Distilling the knowledge from handcrafted features for human activity recognition," *IEEE Trans. Ind. Inform.*, vol. 14, no. 10, pp. 4334–4342, Oct. 2018.
- [19] E. Lattanzi, C. Contoli, and V. Freschi, "Do we need early exit networks in human activity recognition?," *Eng. Appl. Artif. Intell.*, vol. 121, 2023, Art. no. 106035.
- [20] E. Lattanzi, C. Contoli, and V. Freschi, "A study on the energy sustainability of early exit networks for human activity recognition," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 1, pp. 61–74, Jan./Feb. 2024.
- [21] J. Seon, J. Hwang, J. Mun, and B. Han, "Stop or forward: Dynamic layer skipping for efficient action recognition," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2023, pp. 3361–3370.
- [22] G. Yang, L. Zhang, C. Bu, S. Wang, H. Wu, and A. Song, "FreqSense: Adaptive sampling rates for sensor-based human activity recognition under tunable computational budgets," *IEEE J. Biomed. Health Inform.*, vol. 27, no. 12, pp. 5791–5802, Dec. 2023.
- [23] F. Daghero et al., "Human activity recognition on microcontrollers with quantized and adaptive deep neural networks," *ACM Trans. Embedded Comput. Syst.*, vol. 21, no. 4, pp. 1–28, 2022.
- [24] G. Huang and D. Chen, "Multi-scale dense networks for resource efficient image classification," in *Proc. Int. Conf. Learn. Representations*, 2018.

- [25] C. Li, D. Niu, B. Jiang, X. Zuo, and J. Yang, "Meta-HAR: Federated representation learning for human activity recognition," in *Proc. Web Conf.*, 2021, pp. 912–922.
- [26] J. Shu et al., "Meta-Weight-Net: Learning an explicit mapping for sample weighting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1919–1930.
- [27] G. Huang, Z. Liu, L. V. D. Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.
- [28] Z. You, K. Yan, J. Ye, M. Ma, and P. Wang, "Gate decorator: Global filter pruning method for accelerating deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 2133–2144.
- [29] M. Lin, R. Ji, Y. Zhang, B. Zhang, Y. Wu, and Y. Tian, "Channel pruning via automatic structure search," in *Proc. 29th Int. Conf. Int. Joint Conf. Artif. Intell.*, 2021, pp. 673–679.
- [30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2016, pp. 770–778.
- [31] G. Zhao, W. Yang, X. Ren, L. Li, Y. Wu, and X. Sun, "Well-classified examples are underestimated in classification with deep neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 9180–9189.
- [32] F. J. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, 2016, Art. no. 115.
- [33] D. J. Bajpai and M. K. Hanawal, "CEEBERT: Cross-domain inference in early exit BERT," 2024, *arXiv:2405.15039*.
- [34] D. J. Bajpai and M. Hanawal, "DAEE: Unsupervised domain adaptation in early exit PLMs," in *Proc. Findings Assoc. Comput. Linguistics*, Miami, FL, USA, 2024, pp. 6389–6400.
- [35] H. Qian, S. J. Pan, and C. Miao, "Latent independent excitation for generalizable sensor-based cross-person activity recognition," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 11921–11929.
- [36] H. Yao et al., "Improving out-of-distribution robustness via selective augmentation," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 25407–25437.
- [37] W. Lu et al., "Diversify: A general framework for time series out-of-distribution detection and generalization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 6, pp. 4534–4550, Jun. 2024.
- [38] K. Altun, B. Barshan, and O. Tunçel, "Comparative study on classifying human activities with miniature inertial and magnetic sensors," *Pattern Recognit.*, vol. 43, no. 10, pp. 3605–3620, 2010.
- [39] M. Zhang and A. A. Sawchuk, "USC-HAD: A daily activity dataset for ubiquitous activity recognition using wearable sensors," in *Proc. 2012 ACM Conf. Ubiquitous Comput.*, 2012, pp. 1036–1043.
- [40] A. Reiss and D. Stricker, "Introducing a new benchmarked dataset for activity monitoring," in *Proc. IEEE 16th Int. Symp. Wearable Comput.*, 2012, pp. 108–109.